

Trips Report Analysis: Documentation

Author Details:

Name: Akash Kumar

Role: Data Engineer Intern

Project: Trips Report Analysis – End-to-End Data Engineering Pipeline in Microsoft Fabric

Manager: Harshal Thorat

Table of Contents:

- ❖ Executive Summary
- ❖ Introduction
- ❖ Architecture Overview
- ❖ Data Pipeline Details
- ❖ Notebooks and Code Explanation
- ❖ Semantic Models and Data Modeling
- ❖ Dashboards and Visualizations
- ❖ Technical Services in Microsoft Fabric
- ❖ Implementation Challenges and Solutions
- ❖ Conclusion and Recommendations

Executive Summary:

- This documentation covers the capstone projects involving a data pipeline for trip data in Microsoft Fabric.
- The pipeline follows a medallion architecture (Bronze, Silver, Gold) with streaming ETL, data cleaning, dimension/fact tables, semantic models, and Power BI dashboards.
- Key features include real-time data processing, analytics on revenue, miles, and profits, and visualizations for insights. Backend processes use Spark notebooks for transformations, and Fabric services like Lakehouse and Eventstreams for ingestion.
- Dashboards provide interactive reports on trip expenses and journeys. No major issues encountered; the system processes ~5000 trips efficiently.

Project Background:

The projects focus on building an end-to-end data solution for trip data using Microsoft Fabric. Data is ingested in real-time, processed through layers, modeled, and visualized.

Objectives:

- Implement streaming ETL for trip data.
- Clean and transform data for analytics.
- Create semantic models and dashboards for insights.
- Document all features, including Fabric services.

Scope:

Covers ingestion from custom endpoints, Bronze-Silver-Gold processing, star schema modeling, and three dashboards: Trip Expense Navigator, Trips Report Analysis, Journey Analytics.

Architecture Overview:

Fast API Streaming Worker → Fabric Event stream → Bronze Lakehouse → Silver Layer → Gold Layer → Power BI Dashboards Reports

Trip Streaming Data Worker:

A lightweight real-time trip/event data generator built using Fast API that simulates trip events and streams them continuously into Microsoft Fabric Event stream through a custom endpoint.

The streaming data is ingested into the Lakehouse Bronze layer and processed into silver and gold layers following the medallion architecture.

API Endpoints:

1. POST – /stream/start

Description: Start continuous streaming

- curl -X POST "<https://trips-streaming-data.onrender.com/stream/start>"
- curl -X POST "<https://trips-streaming-data.onrender.com/stream/start?duration=3600>"

2. POST – /stream/stop

Description: Stop the running stream

- curl -X POST "<https://trips-streaming-data.onrender.com/stream/stop>"

3. GET – /stream/status

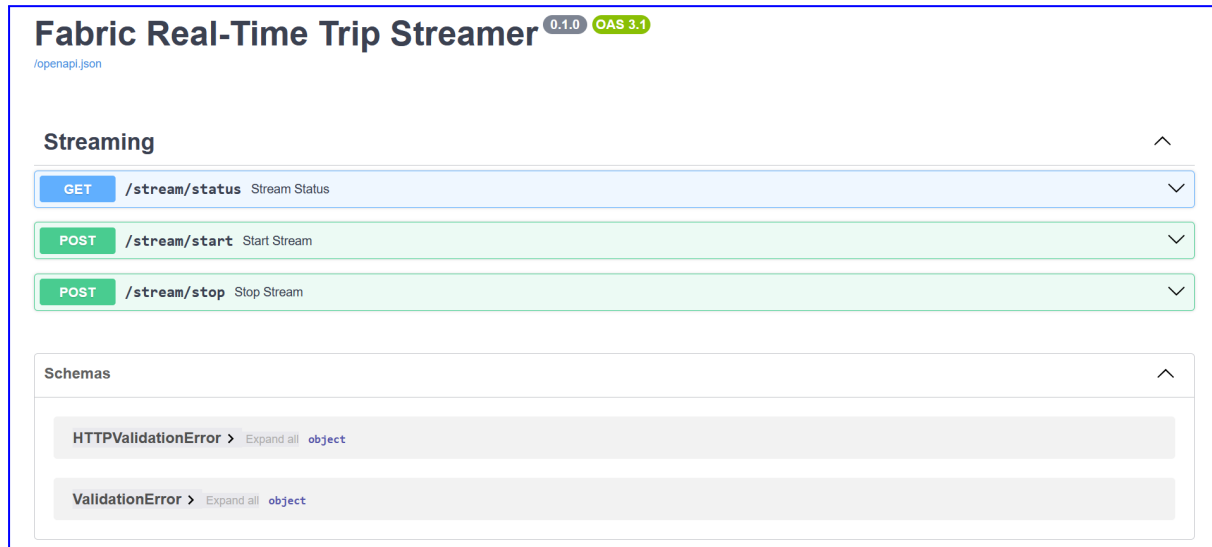
Description: Get current streaming status

- curl "<https://trips-streaming-data.onrender.com/stream/status>"

4. GET – /docs

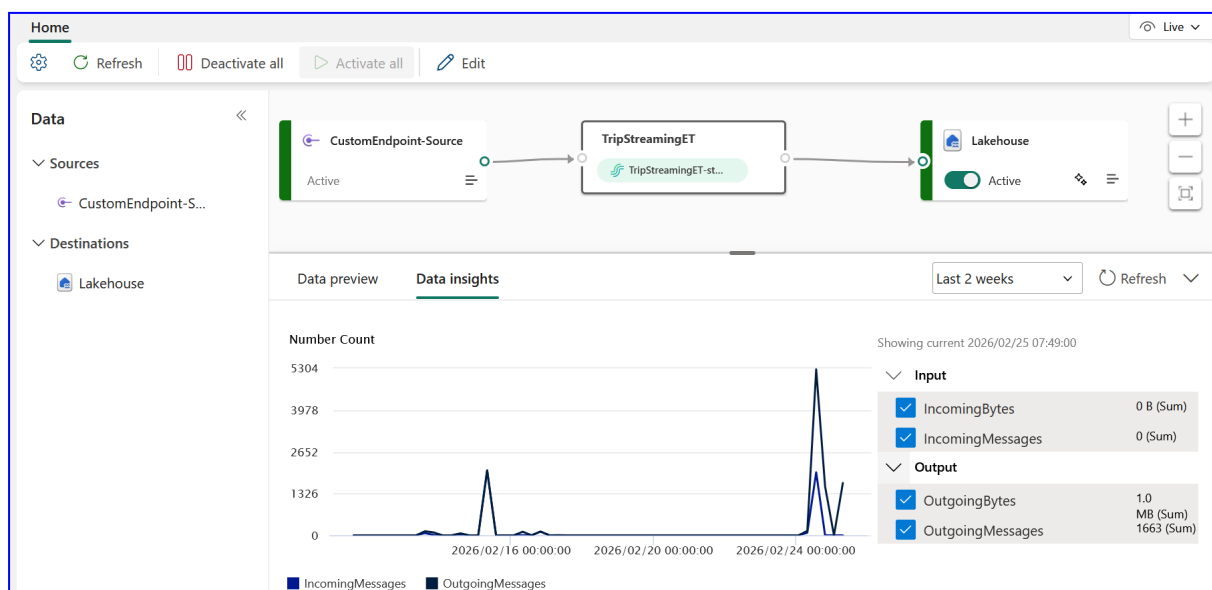
Description: API Interactive Documentation (Swagger UI)

- curl <https://trips-streaming-data.onrender.com/docs>



Features of Streaming Worker:

- Real-time trip event generation
- REST API to start/stop streaming
- Configurable streaming duration
- Fabric-ready JSON schema
- Dockerized deployment



Docker Deployment:

- The entire Trip Streaming Data Worker is containerized using Docker for
- consistent, portable, and easy deployment across environments.
- **Build Image:** `docker build -t trip-streaming-data-worker .`
- **Run Locally:** `docker run -p 8000:8000 --env-file .env trip-streaming-data-worker`
- **Pull and use the public pre-built image :**
 - `docker pull docker.io/akashowebdev/trip-streamerv1:latest`
 - `docker run -p 8000:8000 --env-file .env docker.io/akashowebdev/trip-streamerv1:latest`

Source Code & Image:

Full source code: <https://github.com/AKASH-DEV-23/trip-streaming-data-worker>

Docker image: <https://hub.docker.com/r/akashowebdev/trip-streamerv1>

Medallion Architecture:

- Medallion Architecture is a multi-layered data design that organizes data into three layers—Bronze, Silver, and Gold—to improve data quality, reliability, and usability.
- It's widely used in data lakes and lakehouses to structure ingestion → cleaning → consumption.

1. Bronze Layer:

Purpose: Store raw, unprocessed data exactly as it is received.

Characteristics:

Immutable (append-only) data is never overwritten or modified.

Contains duplicates, errors, and original fields — nothing is cleaned or transformed here.

Used for audit, traceability, and full reprocessing if downstream logic changes.

Ingests data from multiple raw sources, including:

Databases: Streaming source

Custom endpoints integrated via Eventstreams (real-time ingestion)

A SQL analytics endpoint for SQL querying was created with this item.

Explorer

Add lakehouses

- LH_Bronze_Raw
 - Tables
 - dbo
 - realtime_dbo
 - trips_stream_data
 - Files
 - checkpoints

trips_raw

Showing 1000 rows

Table view

	ABC	TripID	ABC	ShipperID	ABC	CategoryID	ABC	Customer	ABC	ShipDate	ABC	OriginCity	ABC	OriginState	ABC
1		10015		2		6		Conzumzap		2016-05-19		Appleton		WI	1
2		10035		2		5		Overtam		2016-06-07		Madison		WI	1
3		10055		2		6		Instrip		2016-02-22		Detroit		MI	1
4		10075		1		4		Vaiazozice		2016-03-11		Springfield		IL	1
5		10095		1		5		Rancan		2016-01-14		Kalamazoo		MI	1
6		10115		2		5		Ronlamice		2016-06-09		Lansing		MI	1
7		10135		1		8		apzone		2016-08-30		Appleton		WI	2
8		10155		1		4		Fasetech		2016-02-18		Madison		WI	2
9		10175		1		6		Faseron		2016-06-05		Detroit		MI	2
10		10195		1		6		Ontohatron		2016-04-22		Springfield		IL	1
11		10215		1		6		Rankphase		2016-08-04		Kalamazoo		MI	2
12		10235		1		6		Fasezoomfix		2016-02-28		Lansing		MI	2
13		10255		1		5		Linezatlam		2016-07-06		Appleton		WI	2

Succeeded (3 sec 523 ms)

Columns 21 Rows 1,000

2. Silver Layer:

- Reads streaming data from the Bronze Delta table.
- Standardizes data types for all numeric, financial, and ID columns.
- Trims whitespace from all string columns.
- Fixes known typos such as converting “Detroit” to “Detroit”.
- Parses ShipDate and DeliveryDate into proper date formats.

Home Edit AI tools Run View

Comments History Develop Share

Run all Connect PySpark (Python) Environment Workspace default Data Wrangler

Explorer

Data items Resources Connections

+ Add data items

- OneLake
 - LH_Silver_Cleaned
 - Tables
 - dbo
 - Realtime
 - trips_cleaned
 - Files

```

48
49 # Trim strings
50 string_columns = [c for c, t in streaming_df.dtypes if t == "string"]
51 for c in string_columns:
52     streaming_df = streaming_df.withColumn(c, trim(col(c)))
53
54
55 # Fix typo
56 if "OriginCity" in streaming_df.columns:
57     streaming_df = streaming_df.withColumn(
58         "OriginCity",
59         when(col("OriginCity") == "Detriot", "Detroit").otherwise(col("OriginCity"))
60     )
61
62
63 # Calculate Profit
64 if all(c in streaming_df.columns for c in ["Profit", "Revenue", "ShippingCost"]):
65     streaming_df = streaming_df.withColumn(
66         "Profit",
67         when(
68             col("Profit").isNull() &
69             col("Revenue").isNotNull() &
70             col("ShippingCost").isNotNull(),
71             col("Revenue") - col("ShippingCost")
72         ).otherwise(col("Profit"))
73     )
74
75

```

Session timeout. Run the notebook to start a new session. AutoSave: On Selected Cell 2 of 5 cells

- Auto-generates DeliveryDate when missing using ShipDate + ShipDays.
- Recalculates Profit when it is null by using Revenue – ShippingCost.
- Renames inconsistent column names like Revenue Miles → Revenue_Miles and Profit miles → Profit_Miles.
- Applies data-quality filters ensuring all numeric fields have non-negative values.
- Ensures schema completeness by adding missing columns with null defaults.
- Performs upsert (MERGE) into the Silver table using TripID as the primary key.
- Uses checkpointing to maintain streaming reliability and exactly-once processing.
- Produces a cleaned, standardized, analytics-ready Silver dataset.

The screenshot shows a PySpark notebook interface. On the left is the 'Explorer' pane showing a project structure under 'OneLake' with a folder 'LH_Silver_Cleaned' containing 'Tables' (with sub-tables 'dbo' and 'Realtime') and 'Files'. The 'Realtime' table has a sub-table 'trips_cleaned'. The main area is a code editor with the following PySpark code:

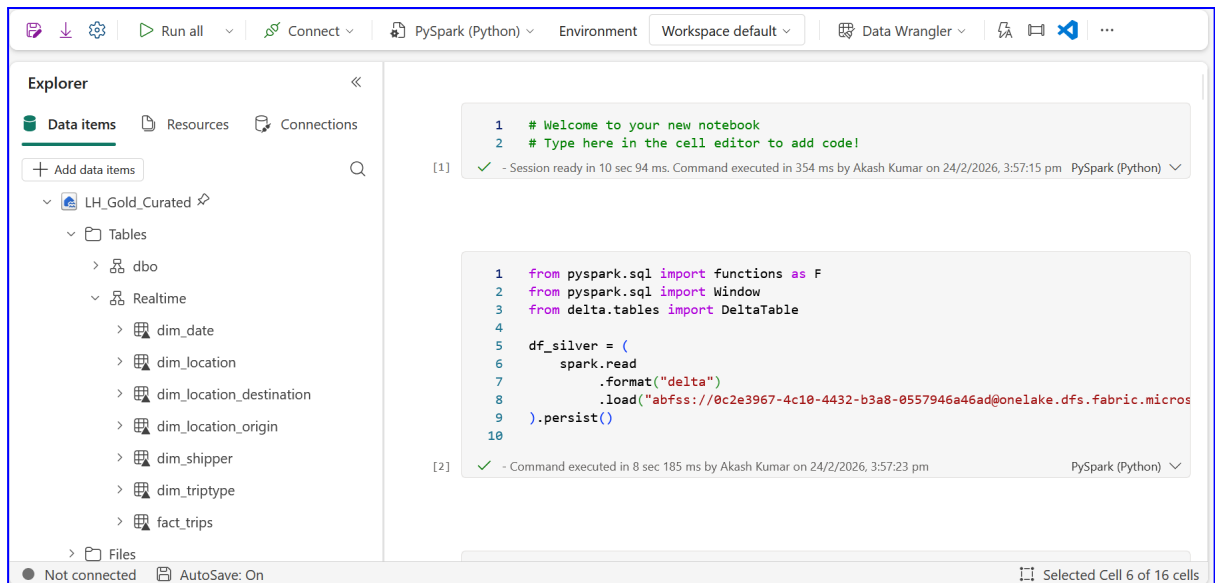
```

1
2
3 from pyspark.sql import functions as F
4 from pyspark.sql.functions import col, trim, when, to_date, lit
5 from delta.tables import DeltaTable
6
7 try:
8     query.stop()
9 except:
10     pass
11
12 # -- Paths
13 BRONZE_PATH = "abfss://0c2e3967-4c10-4432-b3a8-0557946a46ad@onelake.dfs.fabric.microsoft.com/0c2e3967-4c10-4432-b3a8-0557946a46ad/bronze"
14
15 SILVER_TABLE = "LH_Silver_Cleaned.Realtime.trips_cleaned"
16
17 CHECKPOINT_PATH = "abfss://0c2e3967-4c10-4432-b3a8-0557946a46ad@onelake.dfs.fabric.microsoft.com/0c2e3967-4c10-4432-b3a8-0557946a46ad/silver"
18
19 # -- Read stream
20 streaming_df = (
21     spark.readStream
22     .format("delta")
23     .option("maxFilesPerTrigger", 50)
24     .load(BRONZE_PATH)
25 )
26
27 # Type casting

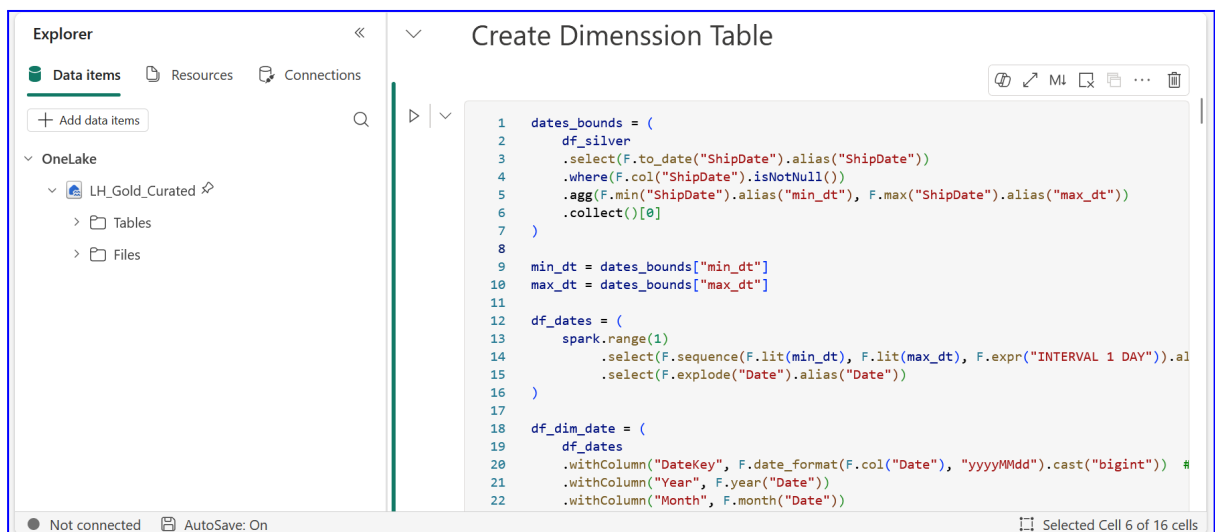
```

3. Gold Layer:

- Reads the cleaned Silver dataset and loads it into memory for transformation.
- Defines a reusable merge function to create or upsert data into Delta tables for incremental updates.
- Normalizes text fields (city, state, customer names, trip types) by trimming spaces, fixing casing, and removing inconsistencies for uniform dimension values.

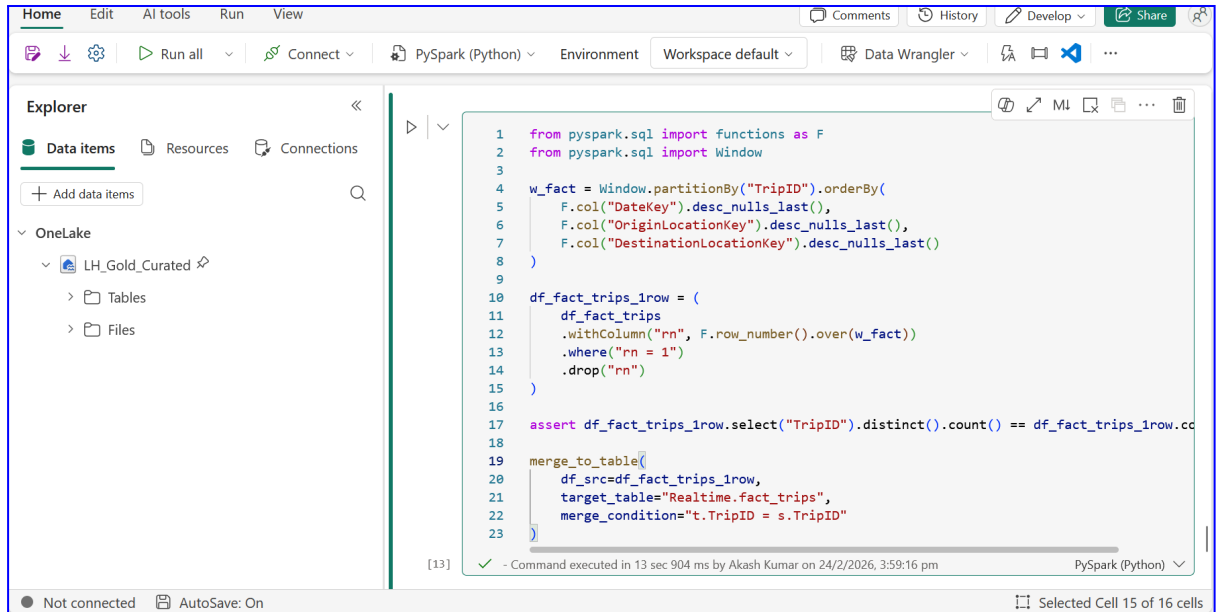


- Generates a complete Date dimension by creating daily records between the minimum and maximum ShipDate values and deriving Year, Month, MonthName, Quarter, and a numeric DateKey.
- Builds a standardized Location dimension by extracting unique origin and destination city/state combinations and generating a surrogate LocationKey using a CRC32 hash.
- Creates separate Origin and Destination Location dimensions for clearer relationships and easier joins in the fact model.
- Creates the Shipper dimension by selecting the best ShipperName per ShipperID using a window function, generating a ShipperKey, and removing duplicates.



- Creates the TripType dimension by normalizing unique TripType values and generating hashed TripTypeKeys.

- Prepares a normalized version of the Silver data with cleaned city/state fields, cleaned TripType, and parsed ShipDate for dimension joins.



```

1 from pyspark.sql import functions as F
2 from pyspark.sql import Window
3
4 w_fact = Window.partitionBy("TripID").orderBy(
5     F.col("DateKey").desc_nulls_last(),
6     F.col("OriginLocationKey").desc_nulls_last(),
7     F.col("DestinationLocationKey").desc_nulls_last()
8 )
9
10 df_fact_trips_1row = (
11     df_fact_trips
12     .withColumn("rn", F.row_number().over(w_fact))
13     .where("rn = 1")
14     .drop("rn")
15 )
16
17 assert df_fact_trips_1row.select("TripID").distinct().count() == df_fact_trips_1row.count()
18
19 merge_to_table(
20     df_src=df_fact_trips_1row,
21     target_table="Realtime.fact_trips",
22     merge_condition="t.TripID = s.TripID"
23 )

```

[13] ✓ - Command executed in 13 sec 904 ms by Akash Kumar on 24/2/2026, 3:59:16 pm PySpark (Python)

Not connected AutoSave: On Selected Cell 15 of 16 cells

- Joins the normalized Silver data to all created dimensions (Date, Origin Location, Destination Location, Shipper, TripType) to attach surrogate keys.
- Constructs the Fact Trips table by selecting required keys (DateKey, OriginLocationKey, DestinationLocationKey, ShipperKey, TripTypeKey) and important measures (Miles, Revenue, Cost, Profit, RevenueMiles, ProfitMiles).
- Writes the fact table as Delta, ensuring a clean overwrite for the initial version.
- Applies row-number based deduplication to retain the most complete/latest record per TripID, ensuring one fact entry per trip.
- Performs a MERGE operation on the fact table so updates or new trips are incrementally added without duplication.
- Produces a fully modeled star-schema dataset with Dimensions + Fact, optimized for BI dashboards, reporting, and semantic modeling.

4. Semantic Model & Data Modeling:

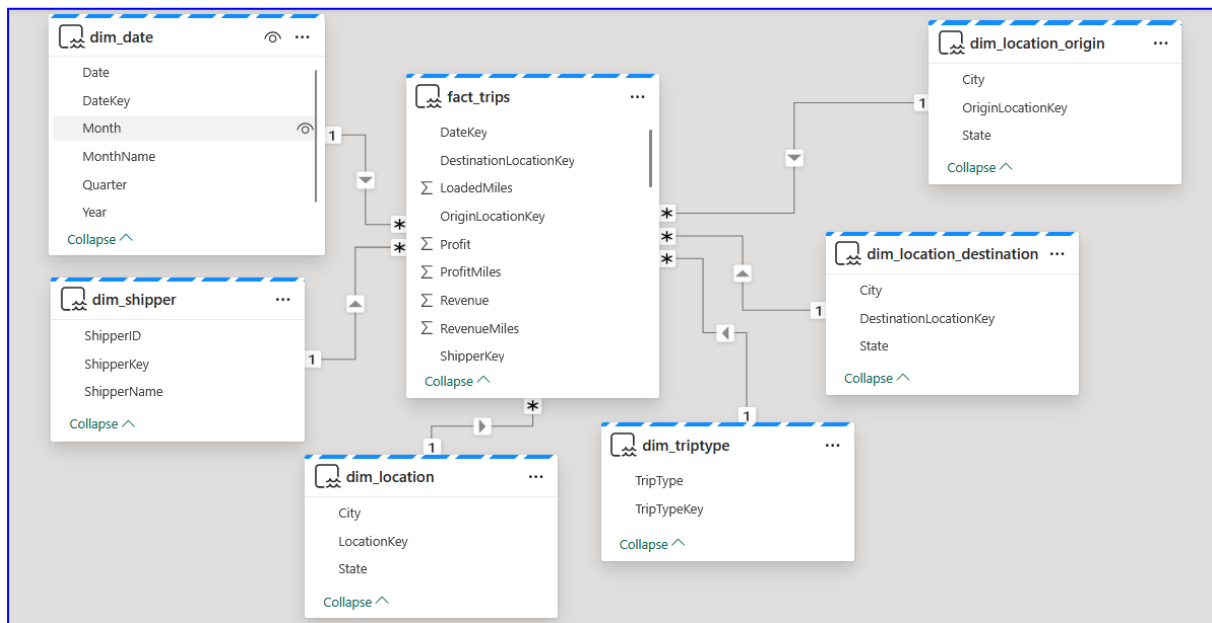
The semantic model represents the business-friendly, analytical layer of the Lakehouse, built on top of the curated Gold tables. It organizes cleaned and transformed data into a star-schema structure, enabling intuitive reporting, optimized performance, and consistent metric definitions across dashboards and analytics tools.

Key Design Principles:

Star Schema Architecture: The model follows a star-schema layout where a single fact table is linked to several dimension tables through surrogate keys. This structure ensures high query performance and simplifies analytical workflows.

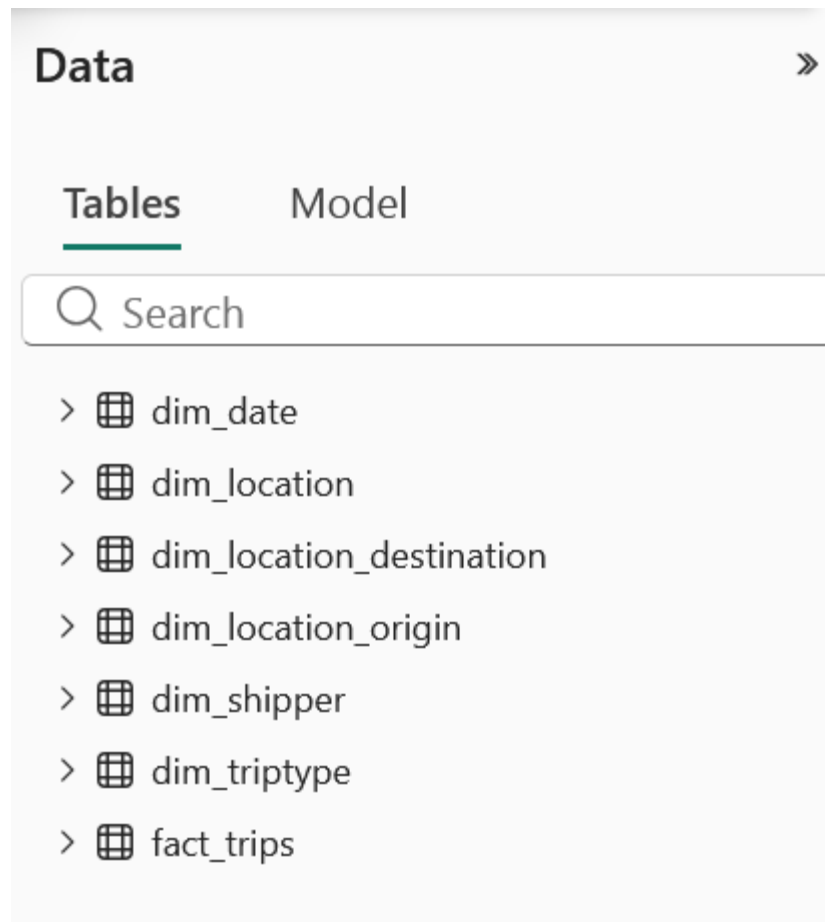
Surrogate Key–Based Relationships: Each dimension uses a stable, system-generated key (DateKey, LocationKey, ShipperKey, TripTypeKey) instead of natural keys. This design prevents inconsistencies, supports incremental loads, and maintains reliable, one-to-many relationships with the fact table.

Business-Friendly Semantic Layer: Technical field names and raw data structures are abstracted into logical business entities such as *Trips*, *Dates*, *Locations*, *Shippers*, and *Trip Types*, making the model easier for analysts and business users to understand.



Relationships & Cardinality:

- Each dimension table has a **1-to-many** relationship with the fact_trips table.
- Relationship direction is enforced from dimension → fact, ensuring clear filtering behavior and optimal performance in BI tools.
- All keys are numeric and surrogate-based, improving compression, join speed, and model stability.



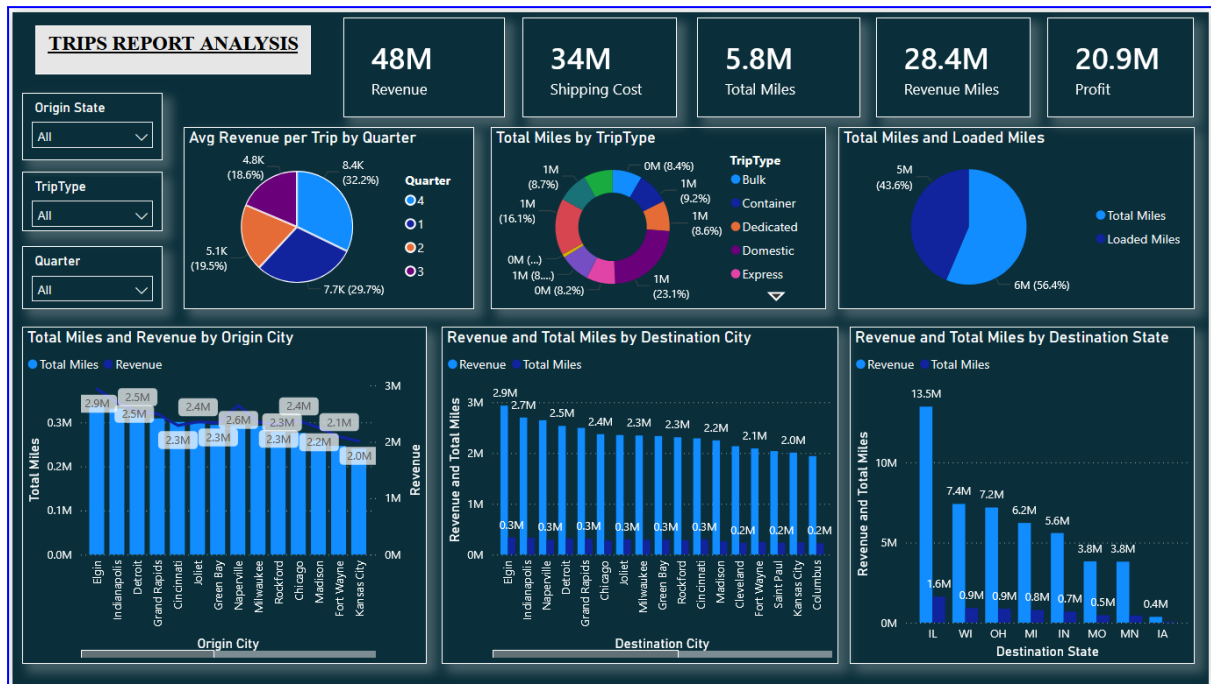
5. Dashboards and Visualizations:

- The project includes a comprehensive suite of interactive dashboards designed to provide actionable insights into trip performance, operational efficiency, revenue trends, and cost management.
- Built on top of the curated Gold semantic model, these dashboards transform raw trip data into clear visual narratives that support decisionmaking across operations, finance, logistics, and performance analysis teams.
- The dashboards use a combination of KPIs, charts, filters, and comparative views to help users analyze trips from multiple dimensions such as time, trip type, shipper, origin/destination, and state-wise performance.

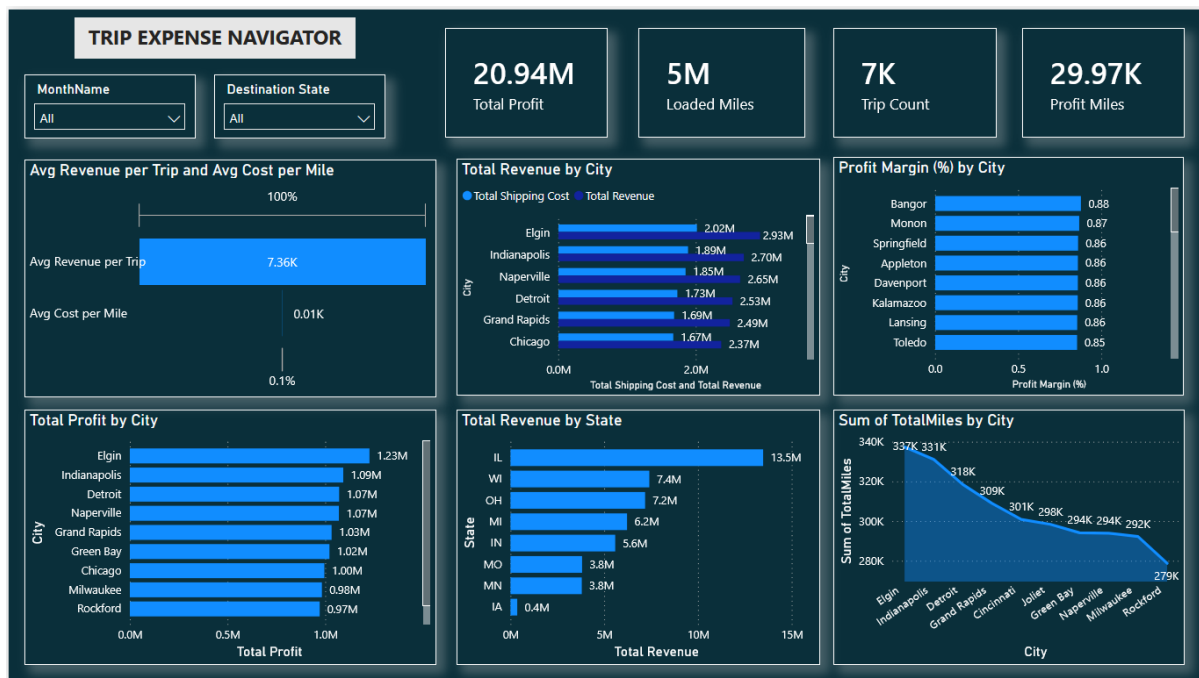
Dashboard Overviews:

- **Trips Report Analysis Dashboard:** This dashboard provides a high-level operational overview of trip performance. It includes
 - Revenue, Shipping Cost, Total Miles, Revenue Miles, and Profit KPIs
 - Revenue per Trip by Quarter
 - Miles distribution by TripType
 - Total Miles vs Loaded Miles comparison

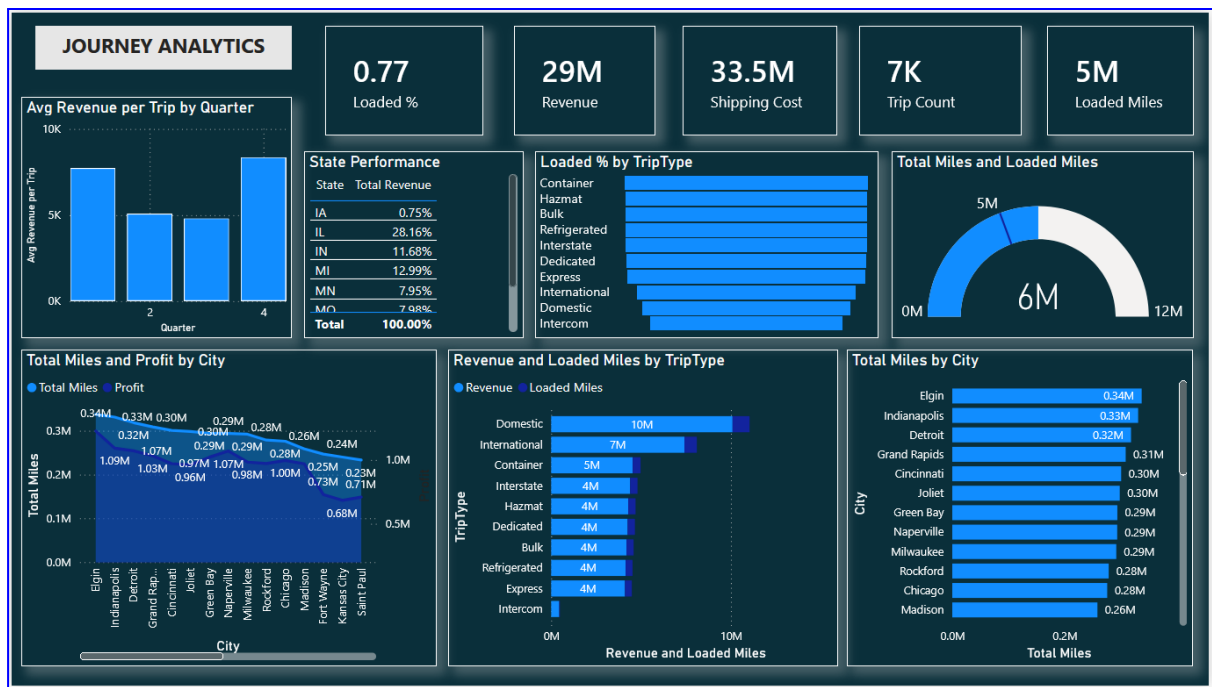
- City-wise and State-wise performance for both origins and destinations
- This view helps identify top cities, high mileage regions, and revenue concentration zones.



- **Trip Expense Navigator:** Designed for cost and profitability analysis, it showcase
 - Total Profit, Loaded Miles, Trip Count, and Profit Miles metrics
 - Avg Revenue per Trip and Avg Cost per Mile comparison
 - Detailed city-wise Revenue and Profit
 - State-wise Revenue distribution
 - Total Miles by City analysis
 - This dashboard helps evaluate cost efficiency, margin performance, and profit-driving locations.



- **Journey Analytics Dashboard:** Focused on deeper operational trends, this dashboard highlights:
 - Avg Revenue per Trip by Quarter
 - State Performance ranking by Total Revenue percentage
 - Loaded Percentage by TripType
 - Total Miles vs Loaded Miles gauge
 - City and TripType-wise revenue and mileage breakdown
 - This helps track seasonal variations, trip efficiency, and route performance across the network.



Purpose and Benefits:

- Converts curated lakehouse data into clear, data-driven stories for stakeholders.
- Enables cross-functional teams to make informed decisions based on real-time, accurate, and high-quality metrics.
- Helps identify revenue opportunities, cost inefficiencies, and operational bottlenecks.
- Supports strategic planning, route optimization, customer profitability analysis, and performance monitoring.

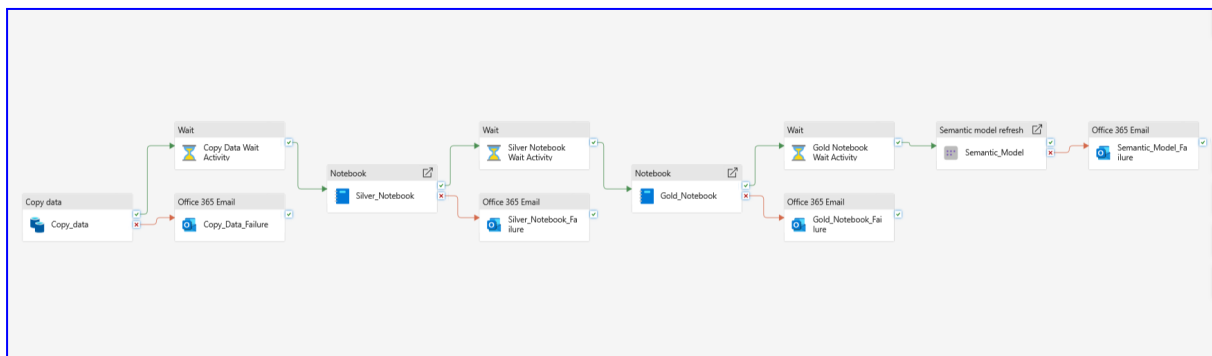
Automated Data Pipeline Orchestration:

This pipeline automates the complete data processing flow from ingestion to dashboard refresh.

Each stage runs sequentially and includes built-in failure handling.

- **Copy Data:**
 - Ingests raw data into the Lakehouse.
 - Begins the Bronze → Silver → Gold processing sequence.
- **Silver Notebook Execution**
 - Runs the Silver transformation notebook after the copy step finishes.
 - Cleans, standardizes, and enriches raw data.
 - Send an email alert if the Silver step fails.

- Gold Notebook Execution
 - Starts once Silver processing is complete.
 - Builds dimensions and fact tables for the Gold layer.
 - Send a failure email if the Gold notebook encounters an error.
- Semantic Model Refresh
 - Refreshes the Fabric/Power BI semantic model to load updated Gold data.
 - Ensures dashboards always show the latest validated information.
 - Sends an alert if the semantic model refresh fails.
- End-to-End Reliability
 - Each step uses wait activities to ensure ordered execution.
 - Email alerts allow quick response to failures.
 - Ensures fully automated and reliable data delivery to dashboards.



Data Masking Implementation:

To protect sensitive financial information within the analytics environment, Dynamic Data Masking (DDM) was implemented on the `fact_trips` table. This ensures that non-privileged users can query the table without seeing actual financial values, improving security and compliance.

Purpose:

- Prevent exposure of sensitive fields (Revenue, ShippingCost, Profit).
- Allow analysts to work with data safely without accessing confidential values.
- Ensure secure access control in shared environments.
- Support regulatory and organizational data-privacy requirements.

Columns Protected with Masking:

The following fields in `fact_trips` are masked to prevent exposure of real financial data:

- Revenue
- ShippingCost
- Profit

These columns appear in masked form for all public or non-authorized roles.

Access Control:

Public users receive read-only access to the table.

Masking ensures that even with read access, sensitive values remain protected.

Only users with explicit UNMASK rights can view actual data.